Autoencoders (Practicals)

Structured Notes

Encoder-Decoder Architectures, U-Net

Segmentation, Simple and
Convolutional Autoencoders, Denoising Autoencoders

Contents

1 Encoder-Decoder Architectures **2**
1.1 Applications of Encoder-Decoder Architectures . . . 2
1.2 How Encoder-Decoder Works . . . 2

2 U-Net Architecture for Image Segmentation **2**
2.1 Problem Setup: Blood Cell Segmentation . . . 2
2.2 Data Preparation . . . 2
2.3 U-Net Architecture . . . 3
2.3.1 Encoder Block (Down Block) . . . 3
2.3.2 Bottleneck Layer . . . 3
2.3.3 Decoder Block (Up Block) . . . 3
2.4 U-Net Implementation in Keras . . . 4
2.5 Model Compilation and Training . . . 4

3 Autoencoders **5**
3.1 Autoencoder vs. Encoder-Decoder . . . 5
3.2 The Latent Vector . . . 5

4 Simple Autoencoder with Dense Layers **5**
4.1 Dataset: MNIST . . . 5
4.2 Architecture . . . 6

5 Convolutional Autoencoder **7**
5.1 Key Differences from Dense Autoencoder . . . 7
5.2 Encoder Architecture . . . 7
5.3 Decoder Architecture . . . 7
5.4 Convolution vs. Transpose Convolution . . . 7
5.5 Implementation . . . 8
5.6 Trainable vs. Non-Trainable Parameters . . . 8

6 Denoising Autoencoders **8**
6.1 What the Latent Vector Contains . . . 9
6.2 Denoising Autoencoder in PyTorch . . . 9
6.3 Architecture Details . . . 9
6.4 Data Loaders . . . 10

7 Keras vs. PyTorch **10**

8 Summary **11**

1 Encoder-Decoder Architectures

Encoder

An **encoder** is a neural network component that takes an input, extracts features from it, and converts it into a **compressed vector** (also called a **latent representation**, **context vector**, or **latent vector**). This compressed vector contains the overall summary information about the input.

Decoder

A **decoder** takes the compressed vector produced by the encoder and generates the desired output. The decoder reconstructs or transforms the latent representation into the target domain.

1.1 Applications of Encoder-Decoder Architectures

Encoder-decoder architectures are used for tasks where input from one domain needs to be transformed into output in another domain:

- **Image reconstruction** — Reconstructing images from compressed representations
- **Machine translation** — Converting text from one language to another (e.g., Hindi to English)
- **Image captioning** — Generating text descriptions from images
- **Audio to text** — Converting speech signals into text
- **Image segmentation** — Generating pixel-wise masks from input images

1.2 How Encoder-Decoder Works

1. The **encoder** takes the input and extracts features from it
2. These features are compressed into a **latent vector** (context vector)
3. The **decoder** takes the latent vector and generates the output in the target domain

Key Distinction

In encoder-decoder architectures, the **input and output domains can differ**. For example, the input can be an image and the output can be text (image captioning), or the input can be text in one language and the output text in another language (machine translation).

2 U-Net Architecture for Image Segmentation

2.1 Problem Setup: Blood Cell Segmentation

The segmentation task involves microscopic blood images:

- **Input images**: Microscopic blood slide images showing cells
- **Ground truth masks**: Binary images where cells are represented in white and the background in black
- The goal is to generate a mask from the input image where only the cells are detected

This has practical applications in medical imaging — pathologists need to count cells, examine cell shapes, and diagnose diseases. Automated segmentation using encoder-decoder architectures makes this process significantly easier, especially in challenging cases with overlapping cells, uneven staining, or varying illumination.

2.2 Data Preparation

Data Loading and Preprocessing

```python
import numpy as np
from PIL import Image
import os

# Mount drive and set paths
image_path = "/path/to/images/"
mask_path = "/path/to/masks/"

# Load training data
# X_train shape: (39, 512, 512, 3) - 39 RGB images of size 512x512
# Y_train shape: (39, 512, 512, 1) - 39 single-channel masks of size 512
    x512
print(X_train.shape)  # (39, 512, 512, 3)
print(Y_train.shape)  # (39, 512, 512, 1)
```

Listing 1: Loading Images and Masks

The dataset contains:

- **39 training images**, each of size 512×512 with 3 channels (RGB)
- **39 corresponding masks**, each of size 512×512 with 1 channel (binary)
- Two folders: `r_images` (input images) and `r_masks` (ground truth masks)

2.3 U-Net Architecture

U-Net is a predefined encoder-decoder architecture shaped like the letter "U". One side of the U serves as the **encoder** (extracts features) and the other side serves as the **decoder** (reconstructs the image).

2.3.1 Encoder Block (Down Block)

Each encoder block consists of:

1. Two **Convolution2D** layers
2. One **MaxPooling** layer

The U-Net uses **four encoder blocks** with increasing filter sizes:

Block	Filters	Layers
Encoder Block 1	16	2 Conv2D + 1 MaxPool
Encoder Block 2	32	2 Conv2D + 1 MaxPool
Encoder Block 3	64	2 Conv2D + 1 MaxPool
Encoder Block 4	128	2 Conv2D + 1 MaxPool

Filter Size Selection

The number of filters typically starts small and increases as the number of layers increases. For U-Net, the filter sizes are predefined. However, in general, the choice depends on the task. Common progressions include: $4 \rightarrow 8 \rightarrow 16$, or $16 \rightarrow 32 \rightarrow 64 \rightarrow 128$.

2.3.2 Bottleneck Layer

The **bottleneck** sits between the encoder and decoder:

- Contains two **Convolution2D** layers with 256 filters
- This is where the **compressed vector** is obtained
- Represents the most compressed form of the input information

2.3.3 Decoder Block (Up Block)

Each decoder block consists of:

1. **Upsampling** layer
2. **Concatenation** with the corresponding encoder block output (skip connection)
3. Two **Convolution2D** layers

The U-Net uses **four decoder blocks** with decreasing filter sizes (reverse of encoder):

Block	Filters	Layers
Decoder Block 1	128	Upsample + Concat + 2 Conv2D
Decoder Block 2	64	Upsample + Concat + 2 Conv2D
Decoder Block 3	32	Upsample + Concat + 2 Conv2D
Decoder Block 4	16	Upsample + Concat + 2 Conv2D

2.4 U-Net Implementation in Keras

U-Net Model Definition

```
from tensorflow import keras
from keras.layers import Conv2D, MaxPooling2D, UpSampling2D
from keras.layers import concatenate, Input

# Filter sizes for each block
f = [16, 32, 64, 128]

# Encoder blocks (down blocks)
# Each: 2x Conv2D + 1x MaxPooling2D
p0, c0 = down_block(inputs, f[0])    # 16 filters
p1, c1 = down_block(p0, f[1])        # 32 filters
p2, c2 = down_block(p1, f[2])        # 64 filters
p3, c3 = down_block(p2, f[3])        # 128 filters

# Bottleneck: 2x Conv2D with 256 filters
bn = bottleneck(p3, 256)

# Decoder blocks (up blocks)
# Each: Upsample + Concatenate + 2x Conv2D
u0 = up_block(bn, c3, f[3])    # 128 filters
u1 = up_block(u0, c2, f[2])    # 64 filters
u2 = up_block(u1, c1, f[1])    # 32 filters
u3 = up_block(u2, c0, f[0])    # 16 filters
```

Listing 2: Building the U-Net Architecture

2.5 Model Compilation and Training

Compiling and Training the U-Net

```
# Compile the model
model.compile(optimizer='adam',
              loss='binary_crossentropy',
              metrics=['accuracy'])

# Train the model
model.fit(X_train, Y_train,
          validation_split=0.1,
          batch_size=2,
          epochs=20)

```

```
# Save the trained model
model.save('unet_model.h5')

# Load and predict on test data
from keras.models import load_model
model = load_model('unet_model.h5')
predictions = model.predict(X_test)
```

Listing 3: Model Compilation, Training, and Prediction

Training Epochs

The number of epochs (20 in this demonstration) should be adjusted until the validation accuracy or loss becomes stable (saturated). The exact number depends on the specific dataset and task.

3 Autoencoders

Autoencoder

An **autoencoder** is a special type of encoder-decoder architecture where the input and output belong to the **same domain**. The goal is to reconstruct the input from its compressed representation. For example, generating an image from an image, where the output should be as close to the input as possible.

3.1 Autoencoder vs. Encoder-Decoder

Aspect	Encoder-Decoder	Autoencoder
Domain	Input and output can be in **different domains**	Input and output are in the **same domain**
Example	Text (Hindi) → Text (English)	Image → Image
Purpose	Transform input to a different representation	Reconstruct the input faithfully
Applications	Machine translation, image captioning, audio-to-text	Image reconstruction, denoising, feature learning

3.2 The Latent Vector

The **latent vector** (also called compressed vector, context vector, or latent representation) is the core of both encoder-decoder and autoencoder architectures:

- Contains the **summary** of the input — the most informative features
- Its size depends on the task: smaller images (e.g., 28×28 MNIST) can use smaller latent vectors (e.g., 64), while more complex tasks require larger ones
- Contains only the **important features** needed to perform the end goal
- Redundant information is discarded during encoding

4 Simple Autoencoder with Dense Layers

4.1 Dataset: MNIST

The MNIST dataset is used for demonstration:

- Images of numerical digits from 0 to 9
- Image size: 28×28 pixels
- Pixel values range from 0 to 255
- Training set: 60,000 images
- Test set: 10,000 images

Normalization

Pixel values are normalized from the range $[0, 255]$ to $[0, 1]$ by dividing by 255. This helps the neural network train more effectively, as smaller values reduce gradient-related problems.

4.2 Architecture

The simple autoencoder uses only **dense (fully connected) layers**:

- **Encoder**: Dense layers that compress the input into a latent vector of size 64
- **Decoder**: Dense layers that reconstruct the output from the latent vector
- The latent vector of size 64 contains 64 values representing the summary of the input

Simple Autoencoder with Dense Layers

```
from tensorflow import keras
from keras.datasets import mnist
import numpy as np

# Load and normalize MNIST data
(x_train, _), (x_test, _) = mnist.load_data()
x_train = x_train.astype('float32') / 255.0
x_test = x_test.astype('float32') / 255.0

# Define encoder
encoder = keras.Sequential([
    keras.layers.Flatten(input_shape=(28, 28)),
    keras.layers.Dense(128, activation='relu'),
    keras.layers.Dense(64, activation='relu')  # Latent vector size = 64
])

# Define decoder
decoder = keras.Sequential([
    keras.layers.Dense(128, activation='relu', input_shape=(64,)),
    keras.layers.Dense(784, activation='sigmoid'),
    keras.layers.Reshape((28, 28))
])

# Create autoencoder
autoencoder = keras.Sequential([encoder, decoder])

# Compile and train
autoencoder.compile(optimizer='adam', loss='mse')
autoencoder.fit(x_train, x_train,
                epochs=30,
                batch_size=256,
                shuffle=True,
                validation_data=(x_test, x_test))

# Predict on test data
decoded_imgs = autoencoder.predict(x_test)
```

Listing 4: Simple Autoencoder Implementation

Key parameters:

- **Latent vector size**: 64 (contains 64 values summarizing the input)
- **Optimizer**: Adam
- **Loss function**: Mean Squared Error (MSE)
- **Epochs**: 30
- **Batch size**: 256

After training, the model reconstructs the input images from the test dataset. The original images can be compared with the reconstructed outputs to evaluate the quality of reconstruction.

5 Convolutional Autoencoder

The same autoencoder task can be performed using **convolutional layers** instead of dense layers. The task remains the same — reconstruct the input image — but the architecture uses convolution operations.

5.1 Key Differences from Dense Autoencoder

- Uses **Conv2D** layers instead of Dense layers for feature extraction
- Uses **Conv2DTranspose** layers in the decoder for upsampling
- Preserves spatial structure of the image during processing
- Can capture local patterns (edges, textures) more effectively

5.2 Encoder Architecture

The encoder consists of **four layers**:

1. **Conv2D** layer with 32 filters
2. **Conv2D** layer with 64 filters
3. **Flatten** layer (converts 2D feature maps to 1D vector)
4. **Dense** layer (produces the latent vector of size 16)

Spatial Dimension Reduction

After convolution operations, the spatial dimensions are reduced. For a 28×28 input with two Conv2D layers, the spatial dimension reduces to 14×14 and then to a smaller size depending on the kernel and stride configuration.

5.3 Decoder Architecture

The decoder reverses the encoding process:

1. **Input** layer accepting the latent vector (size 16)
2. **Dense** layer to expand back
3. **Conv2DTranspose** layer with 64 filters
4. **Conv2DTranspose** layer with 32 filters
5. Output **Conv2DTranspose** layer with 1 filter (single channel output)

5.4 Convolution vs. Transpose Convolution

Conv2DTranspose

Convolution 2D Transpose (also called transposed convolution or deconvolution) performs the reverse operation of regular convolution. While convolution **decreases** the spatial dimensions by

extracting features, transpose convolution **increases** the spatial dimensions to reconstruct the image.

Property	Conv2D	Conv2DTranspose
Spatial dimension	Decreases	Increases
Purpose	Feature extraction	Image reconstruction
Used in	Encoder	Decoder
Operation	Extracts and compresses	Expands and reconstructs

5.5 Implementation

Convolutional Autoencoder

```
from tensorflow import keras
from keras.layers import Conv2D, Conv2DTranspose, Flatten, Dense
from keras.layers import Reshape, Input

latent_dim = 16
layer_filters = [32, 64]

# Encoder
encoder_input = Input(shape=(28, 28, 1))
x = Conv2D(32, (3, 3), activation='relu', padding='same',
           strides=2)(encoder_input)
x = Conv2D(64, (3, 3), activation='relu', padding='same',
           strides=2)(x)
x = Flatten()(x)
latent = Dense(latent_dim)(x)

# Decoder
decoder_input = Input(shape=(latent_dim,))
x = Dense(7 * 7 * 64, activation='relu')(decoder_input)
x = Reshape((7, 7, 64))(x)
x = Conv2DTranspose(64, (3, 3), activation='relu',
                    padding='same', strides=2)(x)
x = Conv2DTranspose(32, (3, 3), activation='relu',
                    padding='same', strides=2)(x)
decoder_output = Conv2DTranspose(1, (3, 3), activation='sigmoid',
                                 padding='same')(x)

# Compile and train
autoencoder.compile(optimizer='adam', loss='mse')
autoencoder.fit(x_train, x_train, epochs=10,
                batch_size=256, validation_data=(x_test, x_test))
```

Listing 5: Convolutional Autoencoder Implementation

5.6 Trainable vs. Non-Trainable Parameters

- **Trainable parameters**: Learnable parameters (weights and biases) that are **updated during the training process** through backpropagation
- **Non-trainable parameters**: Parameters that are **not updated** during training (e.g., batch normalization running statistics, frozen layer weights)

6 Denoising Autoencoders

Denoising Autoencoder

A **denoising autoencoder** is a type of autoencoder that takes a **noisy input** and learns to reconstruct a **clean output**. The latent vector captures only the important features needed to reconstruct the original image, effectively filtering out the noise.

6.1 What the Latent Vector Contains

In a denoising autoencoder, the latent vector contains:

- Only the **important features** that help reconstruct the clean image
- Information that is essential for the end goal (denoising)
- It does **not** capture redundant features or noise
- The encoder learns to separate useful information from noise

Latent Vector Across Tasks

The content of the latent vector changes based on the task:

- **Machine translation**: Contains linguistic features for converting between languages
- **Segmentation**: Contains information to separate foreground from background
- **Denoising**: Contains features to reconstruct the clean image from noisy data

6.2 Denoising Autoencoder in PyTorch

The denoising autoencoder implementation uses PyTorch with linear (dense) layers:

Denoising Autoencoder Architecture

```
import torch
import torch.nn as nn
from torchvision import datasets, transforms
from torch.utils.data import DataLoader

# Load MNIST dataset
transform = transforms.ToTensor()
train_data = datasets.MNIST(root='./data', train=True,
                            download=True, transform=transform)
test_data = datasets.MNIST(root='./data', train=False,
                           download=True, transform=transform)

# Data loaders
batch_size = 128
train_loader = DataLoader(train_data, batch_size=batch_size,
                          shuffle=True)
val_loader = DataLoader(val_data, batch_size=batch_size)
test_loader = DataLoader(test_data, batch_size=batch_size)
```

Listing 6: Denoising Autoencoder in PyTorch

6.3 Architecture Details

Encoder (Linear Layers with ReLU):

$$784 \rightarrow 512 \rightarrow 256 \rightarrow 128 \text{ (latent vector)}$$

Decoder (Linear Layers with ReLU):

$$128 \to 256 \to 512 \to 784$$

Denoising Autoencoder Model

```python
class DenoisingAutoencoder(nn.Module):
    def __init__(self):
        super(DenoisingAutoencoder, self).__init__()
        # Encoder: 784 -> 512 -> 256 -> 128
        self.encoder = nn.Sequential(
            nn.Linear(784, 512),
            nn.ReLU(),
            nn.Linear(512, 256),
            nn.ReLU(),
            nn.Linear(256, 128),
            nn.ReLU()
        )
        # Decoder: 128 -> 256 -> 512 -> 784
        self.decoder = nn.Sequential(
            nn.Linear(128, 256),
            nn.ReLU(),
            nn.Linear(256, 512),
            nn.ReLU(),
            nn.Linear(512, 784),
            nn.Sigmoid()
        )

    def forward(self, x):
        encoded = self.encoder(x)
        decoded = self.decoder(encoded)
        return decoded

model = DenoisingAutoencoder().to(device)
```

Listing 7: Encoder and Decoder with Linear Layers

- The latent vector size is **128** (the output of the last encoder layer)
- The decoder **reverses** the encoder architecture: number of neurons goes from 128 back to 784
- **ReLU** activation is used throughout
- **Sigmoid** activation on the output layer to produce values in $[0, 1]$

6.4 Data Loaders

DataLoader

A **DataLoader** in PyTorch handles:

- Reading data from the dataset
- Loading data into batches of specified size
- Shuffling data before each training epoch
- Feeding batches to the model during training

Three separate data loaders are created for **training**, **validation**, and **test** data.

7 Keras vs. PyTorch

Both frameworks can be used to build the same architectures. The key points:

- **Keras** is an API defined inside TensorFlow that simplifies building, training, and deploying neural network models
- Keras provides predefined operations (e.g., how Conv2D performs convolution) that are imported and used directly
- **PyTorch** offers similar functionality with a slightly different syntax
- The underlying logic and architecture remain the same regardless of the framework
- The choice of framework does not affect the model performance — only the syntax differs

Aspect	Keras/TensorFlow	PyTorch
API style	High-level, simplified	More flexible, explicit
Importing layers	`from keras.layers import Conv2D`	`import torch.nn as nn`
Layer definition	`Conv2D(32, (3,3), activation='relu')`	`nn.Conv2d(1, 32, 3)`
Training	`model.fit()`	Manual training loop

8 Summary

- **Encoder-decoder architectures** use an encoder to extract features into a compressed vector, and a decoder to generate output from that representation
- **U-Net** is a predefined encoder-decoder architecture with four encoder blocks (Conv2D + MaxPool), a bottleneck, and four decoder blocks (Upsample + Concatenate + Conv2D), used for image segmentation tasks
- **Autoencoders** are a special case of encoder-decoders where input and output are in the same domain (e.g., image to image reconstruction)
- **Simple autoencoders** can be built with dense layers for basic reconstruction tasks on datasets like MNIST
- **Convolutional autoencoders** use Conv2D (encoder) and Conv2DTranspose (decoder) layers to preserve spatial structure during encoding and decoding
- **Transpose convolution** (Conv2DTranspose) increases spatial dimensions, reversing the effect of regular convolution
- **Denoising autoencoders** reconstruct clean images from noisy inputs; the latent vector captures only important features while discarding noise
- The **latent vector** content is task-dependent: it always contains the most important features needed to achieve the specific end goal
- Both **Keras** and **PyTorch** can be used to implement autoencoders with only syntax differences